Day34_IntegritySecurity

Q1 — Integrity Fundamentals [2 Marks]

Explain Integrity and Authenticity. Give one example of each.

- **Integrity**: Ensures that data remains accurate, complete and protected from unauthorized modification.
    - **Example**: A student's examination marks cannot be changed without proper authorization.
- **Authenticity**: Confirms that data or a message comes from a genuine and trusted source.
    - **Example**: A digital signature verifies that a software update was published by the legitimate software developer.

Q2 — CIA Triad [2 Marks]

Explain how Integrity differs from Confidentiality.

- Integrity protects data from unauthorized changes and ensures its accuracy.
- Confidentiality prevents unauthorized people from accessing or viewing sensitive information.

For example, integrity ensures examination marks are not modified, while confidentiality ensures only authorized people can view them.

Q3 — Hash Verification [3 Marks]

Explain how SHA-256 can be used to identify whether a file has changed.

1. SHA-256 is a cryptographic hashing algorithm that generates a fixed-length 256-bit hash from a file.
2. A user calculates and records the original file's SHA-256 hash.
3. After modification, the user calculates the hash again. If the hashes differ, the file contents have changed. If they match, the file is likely unchanged.

Q4 — Software Updates [3 Marks]

A desktop application downloads automatic updates without verifying signatures or hashes. Explain the security risk and recommend controls.

Security risks:

- Attackers could replace legitimate updates with malicious software.

- Modified updates could contain malware or backdoors.
- Users may unknowingly install compromised software.

Recommended controls:

- Verify digital signatures before installing updates.
- Use trusted HTTPS download sources.
- Verify file hashes against trusted published hashes.
- Implement secure update mechanisms and access controls.

Q5 — CI/CD Security [3 Marks]

List and explain at least five security controls that can protect a CI/CD pipeline.

1. **Access Control**: Restrict pipeline access to authorized users only.
2. **Multi-Factor Authentication**: Require additional authentication to protect accounts.
3. **Secret Management**: Store API keys, passwords and tokens securely instead of hardcoding them.
4. **Dependency Scanning**: Scan third-party libraries for known vulnerabilities.
5. **Code Review**: Review and approve code changes before merging or deployment.
6. **Artifact Verification**: Verify hashes or digital signatures before deploying software.

Q6 — Insecure Deserialization [2 Marks]

Explain insecure deserialization at a high level. Why should applications avoid blindly trusting serialized data from untrusted sources?

Insecure deserialization occurs when an application processes serialized data from an untrusted source without properly validating it. Attackers may modify serialized data to manipulate application behaviour, bypass security checks or execute malicious code.

Applications should validate and safely process serialized data instead of blindly trusting user-controlled input.

Q7 — Practical Lab [3 Marks]

Provide evidence from the Day 34 SHA-256 integrity lab. Show Original Hash and Modified Hash, and explain why they differ.

Kali Linux:

Day34_IntegritySecurity

Original Hash:

```
┌──(kirthikaa㉿kali)-[~/Desktop]
└─$     sha256sum day34_original.txt
bc04e4ce99455d1aa7677be5ce79f07e20037f43d70fa05db77416115fa16a30  day34_original.txt
```

Modified Hash:

```
┌──(kirthikaa㉿kali)-[~/Desktop]
└─$ sha256sum day34_original.txt
46dbe19421721a17042be15f53b4fb7157b17ebad9435dc687a9375e996e84f3  day34_original.txt
```

Windows:

Original hash:

```
C:\Users\Kirthikaa\Desktop>certutil -hashfile day34_original.txt SHA256
SHA256 hash of day34_original.txt:
a5eba6eab4662efacc6bdac7e72c158d00ef6a2e44becf8adf5208798dcbbcbc
CertUtil: -hashfile command completed successfully.
```

Modified Hash:

```
C:\Users\Kirthikaa\Desktop>certutil -hashfile day34_original.txt SHA256
SHA256 hash of day34_original.txt:
2f224908ae44cb35b75d718d2b21702e7ed04602ddd641dafba8266cec165fac
CertUtil: -hashfile command completed successfully.
```

- A hash is generated based on the contents of a file. When the text file is modified, its contents change, resulting in a different SHA256 hash.

Q8 — Secure Design [2 Marks]

Provide four recommendations for protecting software/data integrity in a modern application.

- Use SHA-256 hashes and digital signatures to verify file integrity and authenticity.
- Download software and dependencies only from trusted repositories.
- Implement access controls and permissions to prevent unauthorized modifications.
- Secure CI/CD pipelines through code reviews, dependency scanning and artifact verification.